手撕 AI 题详解 —— C++ 版

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 面向：**C++ 强、Python 弱的候选人**。全文所有代码都是 C++，可直接编译运行，已通过 26 项断言。 配套可运行文件：`ai_interview_cpp.cpp`（同目录）， `g++ -O2 -std=c++17 ai_interview_cpp.cpp -o ai && ./ai` 。 阅读方式：第 0、1 章必读（决定你现场怎么写）；第 2 章每题先背"口述脚本"，再背代码骨架，最后扫一遍追问。

---

0. 先解决策略问题：手撕 AI 题到底用什么语言

0.1 结论：用 C++，并且是优势，不是妥协

三个理由，你可以直接说给面试官听：

1. **公式看得见**。`np.dot` 一行看不出你到底懂不懂，而 C++ 里 `s += Q[i*D+t] * K[j*D+t]` 三次循环把"q 与 k 做点积、第 t 维累加"写在脸上——面试官要判断的正是这个。
2. **不依赖环境**。现场共享屏幕，对方可能只给你一个空白编辑器。C++ 只要 `#include <bits/stdc++.h>`，纯 STL 就能跑；torch 版你还要祈祷对方环境里有 GPU/正确版本。
3. **和你的机试语言统一**。华为机试 3 题你本来就用 C++，两套语言切换是额外风险。

开场话术（记住，一句话）：

> "手撕我用 C++ 写，纯 STL，这样每一层矩阵乘和 softmax 的维度我都能显式展开，方便您看我理解得对不对。"

这句话把"我不会 Python"翻译成了"我选择更透明的实现"。绝大多数面试官会说"可以，随你"。

0.2 三个风险点与应对（务必心里有数）

风险	触发场景	应对
面试官坚持要 PyTorch 版	对方团队全是 python 栈	用 4.2 节那三段"可背版"（MHA / softmax+CE / kmeans 向量化），只说"API 层面的封装我知道怎么写，核心逻辑和刚才 C++ 里一模一样"
C++ 写太长，15 分钟写不完	你没提前压缩代码	用第 1 章的"扁平化 + matmul 工具函数"，把 MHA 压到 45 行内；CART 压到 45 行内
边界写崩（数组越界/维度算错）	现场紧张	每道题开头先写 3 行尺寸变量（`int L, D, dk;`），再写循环，最后心里过一遍 `[i*cols+j]` 有没有越界

0.3 谁会被问哪道题（按岗位猜）

题	被问概率	谁爱问
softmax / 交叉熵	极高（几乎是开场题）	所有
self-attention / MHA	极高（多模态岗的核心）	所有
kmeans	中高（聚类、数据清洗、量化里都用）	数据侧 / 端侧
分类树 CART	中（传统 ML 基础题）	偏算法基础的老面试官
IoU / NMS	中高（你有检测背景，必被顺手问）	感知方向
LayerNorm / BN	中（问 Transformer 时连带）	大模型方向

---

1. 通用脚手架：写一次，六道题都复用它

1.1 两个约定（这是你比 Python 选手省时间的地方）

约定 A：矩阵一律用一维 `vector<float>` 扁平存，行主序，`A(i,j) = A[i*cols + j]`。

为什么不用 `vector<vector<float>>`： - 内存连续，cache 友好（讲解时可以说这句，显得懂工程） - 传参只传一个对象 + 两个尺寸，不会写出 `A[i][j]` 忘了 `.size()` - 面试现场最省时间：一行 `vector<float> A(n*m)` 就建好了

约定 B：所有尺寸用 `int` 显式传参，绝不在循环里现算。

例：`matmul(A, B, n, k, m)` 里的 n/k/m 就是 A 的行、A 的列(=B 的行)、B 的列。写代码时先把这三个数在纸上（或注释里）标出来，再动笔。

1.2 工具箱函数 matmul（六道题里四道要用）

```
// A: n x k, B: k x m  ->  C: n x m
vector<float> matmul(const vector<float>& A, const vector<float>& B, int n, int k, int m) {
    vector<float> C(n * m, 0.f);
    for (int i = 0; i < n; ++i)
        for (int t = 0; t < k; ++t) {
            float a = A[i * k + t];
            if (a == 0.f) continue;
            for (int j = 0; j < m; ++j) C[i * m + j] += a * B[t * m + j];
        }
    return C;
}
```

逐行拆解（面试官常让你边写边讲，这段照读就行）：

- `C(n*m, 0.f)`：结果矩阵先全零。**注意必须先初始化**，否则 `+=` 会累加到垃圾值上——这是最常见的现场事故。
- 三层循环 `i → t → j`：C 的每个元素 `C[i][j] = Σ_t A[i][t]·B[t][j]`。**为什么要 i-t-j 而不是 i-j-t**：i-t-j 顺序下，内层 `j` 循环访问的 `C[i*m+j]` 和 `B[t*m+j]` 都是连续内存，缓存命中高；i-j-t 顺序会把 B 每隔 m 个跳着读。面试时补一句"这里我选择 i-t-j 以保持访存连续"，是加分项。
- `if (a == 0.f) continue;`：稀疏跳零，同时避免 `0 * NaN`。
- 索引 `A[i*k+t]`、`B[t*m+j]`、`C[i*m+j]`：**这三个式子写错一个就全错**，背口诀"自己那行的步长是别人列数"。

1.3 三个救命小工具（面试不写，但你本地要练）

```
void dump(const vector<float>& M, int rows, int cols, const char* name) {
    printf("%s [%dx%d]:\n", name, rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) printf("%8.4f", M[i * cols + j]);
        printf("\n");
    }
}
// 随机填充 / 求和校验
vector<float> rnd(int n, unsigned seed);
float sumAll(const vector<float>& v);
```

本地练习法：每次写完一题，`dump` 出来看一眼形状对不对——比肉眼数下标快十倍。现场没有 dump 就口头报形状："Q 是 L×D，S 是 L×L，输出 L×D"。

---

2. 逐题详解

> 每题的固定六段：**题目原话 → 30 秒口述脚本 → 公式 → 代码 → 逐行拆解 → 追问 Q&A → 现场最易错 3 点**。 时间紧就先背"口述脚本"和"最易错 3 点"，这两块决定面试观感。

2.1 Softmax（开场题，必须 3 分钟写完）

面试官原话："手写一个 softmax。"（可能加一句"注意数值稳定"）

30 秒口述脚本：

> "softmax 是三件事：先减最大值防 exp 溢出，再 exp，最后除以总和归一化。减 max 不改变结果，因为分子分母同时乘了 e^{-max}。向量版一遍循环就够，矩阵版逐行调用。"

公式：

```
softmax(x)_i = e^x_i / Σ_j e^x_j = e^(x_i - m) / Σ_j e^(x_j - m),   其中 m = max_j x_j
```

代码（原地版 + 返回版，两个都准备）：

```cpp
// 对 x[off .. off+len-1] 原地做 softmax
void softmaxInPlace(vector<float>& x, int off, int len) {
    float m = -1e30f;
    for (int i = 0; i < len; ++i) m = max(m, x[off + i]);   // 1) 找最大值
    float s = 0.f;
    for (int i = 0; i < len; ++i) {                          // 2) exp(x-m) 并累加
        x[off + i] = expf(x[off + i] - m);
        s += x[off + i];
    }
    for (int i = 0; i < len; ++i) x[off + i] /= s;           // 3) 归一化
}

vector<float> softmax(const vector<float>& x) {
    vector<float> y = x;
    softmaxInPlace(y, 0, (int)y.size());
    return y;
}
```

逐行拆解：

行	在干什么	为什么这么写
`float m = -1e30f;`	用一个足够小的初值	**不要用 `-FLT_MAX`**，因为 `-FLT_MAX - (-FLT_MAX)` 可能出 NaN；初值取 -1e30 是工业惯例
`m = max(m, x[off+i])`	一遍扫出最大值	`std::max_element` 也行，但带偏移量的循环更通用（矩阵逐行调用时直接传 off）
`expf(x-m)`	减 max 后取指数	若 x 全为 1e3，不减 max 会 `exp(1000)` → `inf`，再除 → `nan`
`s += ...`	累加分母	用 `float` 够；若 logits 动态范围极大，可换 `double` 累加（补一句显得严谨）
`/= s`	归一化	结果和恒为 1（浮点误差内）
`off, len` 参数	支持矩阵逐行	这样 MHA 里 `softmaxInPlace(S, i*L, L)` 一行搞定，不用另写矩阵版

追问 Q&A（必背）：

- **Q：为什么减 max 不改变结果？** A：分子分母同乘 e^(-m) 就约掉了：e^(x_i-m) / Σ_j e^(x_j-m) = [e^x_i · e^(-m)] / [Σ_j e^x_j · e^(-m)] = e^x_i / Σ_j e^x_j。
- **Q：减 max 足够吗？还有什么更稳的？** A：减 max 解决上溢；**下溢**（某些项 exp 后为 0）在 softmax 里无害，但在 log 域有害，所以交叉熵要在 log 域用 logsumexp 算（见 2.2）。
- **Q：softmax 的温度 T？** A：指数里除以 T，即 e^(x_i / T)。T>1 分布变平（蒸馏、采样多样性）；T<1 变尖（接近 argmax）；T→0 退化为 one-hot。
- **Q：带 mask 的 softmax 怎么写？** A：把屏蔽位置置成 `-1e30`（C++）或 `float('-inf')`（Python）。**坑**：如果一行全被屏蔽，分母为 0 → NaN。工业实现会给被屏蔽行补 0 权重或直接跳过计算。
- **Q：softmax 和 sigmoid 的关系？** A：二分类时 softmax 两输出等价于 sigmoid（相差一个常数偏移）；多标签（不互斥）用逐元素 sigmoid，对应 BCEWithLogits。
- **Q：softmax 加交叉熵的梯度是什么？**（高频延伸） A：对 logit 求导就是 p_i - y_i，即"预测减标签"，极其干净——这也是为什么分类头后面直接接 CE。

现场最易错 3 点：1. 忘了初始化累加变量（`float s;` 而不是 `float s = 0.f;`）。2. 用 `int s` 累加概率 → 全变 0。3. 减 max 写在除法之后（顺序错）。

2.2 交叉熵 / log_softmax（和 softmax 连着问）

面试官原话："交叉熵损失怎么写？为什么分类任务不直接用 MSE？"

30 秒口述脚本：

> "单样本交叉熵就是 logsumexp(logits) 减掉正确类的 logit。我不先算 softmax 再取 log，而是直接在对数域算，避免概率下溢。数学上是 $-\log p_y$，梯度是 $p - y$。"

公式：

```
L = -log( e^z_y / Σ_j e^z_j ) = logsumexp(z) - z_y,   其中 logsumexp(z) = log Σ_j e^z_j
```

代码：

```cpp
float crossEntropy(const vector<float>& logits, int target) {
    float m = *max_element(logits.begin(), logits.end());
    double s = 0;
    for (float v : logits) s += exp((double)v - m);
    float logsumexp = m + (float)log(s);
    return logsumexp - logits[target];      // = logsumexp(logits) - logits[target]
}
```

逐行拆解：- `m`：再减一次 max，保证 `exp` 不溢出（同 softmax）。- `double s`：累加用 double，减少长类别数下的精度损失（一句话加分项）。- `logsumexp = m + log(s)`：这是恒等式 log Σ e^z_j = m + log Σ e^(z_j - m)，**背下来**，面试官问"logsumexp 技巧"多半就是这句。- `- logits[target]`：分子是正确类，-log(e^z_y / Z) = log Z - z_y。- **没有除以 N**：这是我故意写单样本版，批量版本在外面 `sum / N` 或按类别权重加权。

追问 Q&A：- **Q：为什么不用 MSE 做分类？** A：① 交叉熵配 softmax 的梯度是 `p - y`，误差大时梯度大，收敛快；MSE 配 softmax 的梯度含 p(1-p) 因子，饱和区梯度消失。② MSE 对概率分布不是合适的距离（非凸、非概率性）。- **Q：标签平滑怎么改？** A：target 从 one-hot 换成 (1-ε) 的 one-hot + ε/C 均匀分布；等价的简化实现是 L = (1-ε)·CE + ε·mean(-z)。- **Q：多标签 / 带类别权重？** A：多标签用逐元素 `log(1+e^(-z)) - y·z`（BCEWithLogits）；带权重按类别频率取倒数或 sqrt(1/f)，在分子求和后按权重归一。- **Q：你真实用过的坑？**（这是你的强项，主动讲）A：把"CE target 越界"讲出来——参数名 `cls_num` 与实际类别数不一致导致 target 超出范围、loss 直接崩，属于"配置口径"问题，不是模型问题。

现场最易错 3 点：忘了减 `m`；用 `log(p)` 二次计算（p 可能为 0 → -inf）；偏移 target（写成 0-based/1-based 混淆）。

2.3 LayerNorm（问 Transformer 必连带）

面试官原话："写个 LayerNorm" / "BN 和 LN 的区别，为什么 Transformer 用 LN？"

30 秒口述脚本：

> "LayerNorm 对**单样本的特征维**做归一化：先算这一行所有元素的均值和方差，标准化后再乘可学习的 γ、加 β。和 BatchNorm 的区别是统计量在哪个维度求——BN 跨 batch 求，LN 在样本内求，所以 LN 不受 batch 大小影响，也能用在变长序列上。"

公式：

```
y = γ · (x - μ) / sqrt(σ² + ε) + β,   μ、σ² 是这一行（最后一个维度）的均值方差
```

代码：

```cpp
void layerNorm(vector<float>& x, const vector<float>& g, const vector<float>& b, float eps = 1e-5f) {
    int D = (int)g.size();
    for (size_t row = 0; row + D <= x.size(); row += D) {
        float mu = 0;
        for (int i = 0; i < D; ++i) mu += x[row + i];
        mu /= D;
        float var = 0;
        for (int i = 0; i < D; ++i) { float d = x[row + i] - mu; var += d * d; }
        var /= D;
        float inv = 1.0f / sqrtf(var + eps);
        for (int i = 0; i < D; ++i) x[row + i] = g[i] * (x[row + i] - mu) * inv + b[i];
```

```
    }
}
```

逐行拆解：- `for (row = 0; row + D <= x.size(); row += D)`：把扁平数组按行切分。**注意终止条件是 `row + D <= size`**，不是 `row < size`，否则最后一轮越界——这是 C++ 手写最常见的崩法。- `mu /= D` / `var /= D`：用总体方差（除 D），不是无偏方差（除 D-1）。**框架实现是除 D**，答错会被追一句"你确定？"。- `inv = 1/sqrt(var + eps)`：先算倒数再乘，比每维做一次除法快（这是工程细节加分项）。- `eps=1e-5`：防 var=0 时除零。**注意 eps 在 sqrt 里面**，不是外面。- γ、β 是可学习参数：γ 初始化为 1、β 初始化为 0 时，LN 是恒等变换（这条对"为什么 LN 不破坏表达力"很好用）。

追问 Q&A：- **Q：LN vs BN？** A：BN 对每个通道跨 (N,H,W) 求统计量 → 依赖 batch，小 batch 不稳，推理时要用滑动平均；LN 对单样本所有通道求 → 与 batch 无关，适合变长序列/RNN/Transformer。CV 里小 batch 的替代是 GroupNorm。- **Q：为什么 Transformer 用 Pre-LN 而不是 Post-LN？** A：Post-LN（原始）残差后再归一，深层时梯度需要穿过 LN，必须 warmup 才稳；Pre-LN 把 LN 放在子层输入侧，残差路径是恒等的，梯度直通，可去 warmup，这是现在的主流。- **Q：RMSNorm 是什么？** A：去掉均值中心化，只除 RMS：$y = \gamma \cdot x / \sqrt{\text{mean}(x^2) + \varepsilon}$。少一次求均值和减法，更快，LLaMA 系在用。- **Q：BN 推理时怎么算？** A：用训练期累积的滑动平均 μ、σ^2（momentum 一般 0.1），不再用当前 batch 统计量。这也是"训练/推理不一致"的经典 bug 来源。

现场最易错 3 点：除 D-1；eps 放错位置；`row + D <= size` 写成 `row < size`。

2.4 IoU + NMS（你有检测背景，顺手就问）

面试官原话："写个 IoU" → "那 NMS 呢" → "NMS 有什么问题，怎么改进？"

30 秒口述脚本：

> "IoU 是交集面积除并集面积，注意并集要减掉一次交集，宽高要 clamp 到 0 以上防负数框。NMS 是按分数降序，贪心保留当前最高分，把它 IoU 超阈值的框全部压掉，循环到没有框。"

代码：

```
struct Box { float x1, y1, x2, y2, score; };

float iou(const Box& a, const Box& b) {
    float ix1 = max(a.x1, b.x1), iy1 = max(a.y1, b.y1);
    float ix2 = min(a.x2, b.x2), iy2 = min(a.y2, b.y2);
    float iw = max(0.f, ix2 - ix1), ih = max(0.f, iy2 - iy1);
    float inter = iw * ih;
    float ua = (a.x2 - a.x1) * (a.y2 - a.y1) + (b.x2 - b.x1) * (b.y2 - b.y1) - inter;
    return ua > 0 ? inter / ua : 0.f;
}

vector<int> nms(vector<Box> bs, float thr) {
    sort(bs.begin(), bs.end(), [](const Box& p, const Box& q) { return p.score > q.score; });
    vector<int> keep;
    vector<char> dead(bs.size(), 0);
    for (size_t i = 0; i < bs.size(); ++i) {
        if (dead[i]) continue;
        keep.push_back((int)i);                      // 当前分最高的留
        for (size_t j = i + 1; j < bs.size(); ++j)
            if (!dead[j] && iou(bs[i], bs[j]) > thr) dead[j] = 1;
    }
    return keep;
}
```

逐行拆解：- 交点坐标 = `max(左上)，min(右下)`——两个框的左上取大的、右下取小的，交集才存在。- `max(0.f, ix2-ix1)`：两框不相交时 `ix2-ix1` 为负，不 clamp 会算出负面积 → 负 IoU。**这是最经典的错**。- `ua = area_a + area_b - inter`：并集容斥，**别忘减 inter**。- `ua > 0 ? ... : 0.f`：两个退化框（面积为 0）时防除零。- NMS 用 `dead` 标记数组而非 erase：避免 $O(n^2)$ 的搬移，且保留下标与输入的对应关系（面试官会问"返回的下标怎么对上原数组"）。- `sort` 的 lambda 用 `>` 实现降序；**比较函数必须是严格弱序**（不能写 `>=`，会崩）。

追问 Q&A：- **Q：NMS 的复杂度？** A：朴素 $O(n^2)$；实际用按 score 排序后只跟 top-k 比，或按空间分桶/网格先把候选缩到邻域内，接近 O(n log n)。- **Q：NMS 有什么缺陷？** A：① 密集场景（人群、并排车辆）会误压真实目标；② 阈值敏感；③ 不可导、不能端到端训练。- **Q：改进方案？（必背三个）** A：**Soft-NMS**（不直接删，按 IoU 高斯/线性衰减分数，保留重叠目标）；**IoU-aware / GIoU-DIoU 类 NMS**（用 IoU 预测质量修正分数）；**可学习的 NMS / set prediction**（DETR 用匈牙利匹

配 + 二分图匹配，彻底不用 NMS）。 - **Q：IoU 的局限？** A：不相交时 IoU 恒为 0，无梯度 → GIoU（加最小闭包面积）、DIoU（加中心距离）、CIoU（再加速度一致性），这是检测损失的标准演进线。

现场最易错 3 点：忘 clamp；并集忘减交集；`sort` 比较函数写 `>=`。

2.5 K-Means（考"你会不会讲算法缺陷"多于考代码）

面试官原话："手写一个 kmeans" → "k 怎么选" → "初始化有什么讲究" → "它一定会收敛吗？"

30 秒口述脚本：

> "kmeans 是 EM 的特例：E 步把每个点分给最近的中心，M 步把中心更新成簇内均值，交替到中心移动量小于阈值。复杂度 O(n・k・d・iters)。它只能收敛到局部最优，对初始化和异常值敏感，所以工程上要跑多次初始化取 inertia 最小的，或者用 k-means++。"

代码：

```
vector<int> kmeans(const vector<float>& X, int n, int d, int k,
                   int iters = 100, float tol = 1e-4f, unsigned seed = 42) {
    mt19937 rng(seed);
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    shuffle(idx.begin(), idx.end(), rng);
    vector<float> C(k * d);
    for (int j = 0; j < k; ++j)
        for (int t = 0; t < d; ++t) C[j * d + t] = X[idx[j] * d + t];   // 随机选 k 个样本当初始中心

    vector<int> lab(n, 0), cnt(k, 0);
    for (int it = 0; it < iters; ++it) {
        for (int i = 0; i < n; ++i) {                   // E 步：每个点归到最近中心
            float best = 1e30f;
            int bj = 0;
            for (int j = 0; j < k; ++j) {
                float dist = 0;
                for (int t = 0; t < d; ++t) { float df = X[i * d + t] - C[j * d + t]; dist += df * df; }
                if (dist < best) { best = dist; bj = j; }
            }
            lab[i] = bj;
        }
        vector<float> S(k * d, 0.f);
        fill(cnt.begin(), cnt.end(), 0);
        for (int i = 0; i < n; ++i) {                    // M 步：重新算中心
            int j = lab[i];
            ++cnt[j];
            for (int t = 0; t < d; ++t) S[j * d + t] += X[i * d + t];
        }
        float shift = 0;
        for (int j = 0; j < k; ++j) {
            if (cnt[j] == 0) continue;                   // 空簇：保留旧中心
            for (int t = 0; t < d; ++t) {
                float nc = S[j * d + t] / cnt[j];
                float df = nc - C[j * d + t];
                shift += df * df;
                C[j * d + t] = nc;
            }
        }
        if (sqrtf(shift) < tol) break;                   // 收敛判据：中心移动量
    }
    return lab;
}
```

逐行拆解： - `iota` + `shuffle`：造 0..n-1 的下标再打乱，取前 k 个当初始中心。**注意是在样本里抽，不是随机生成坐标**（随机生成坐标可能离数据很远，收敛慢）。 - E 步用**距离平方**比较：不用开方，省一个 sqrt，且 argmin 结果相同（这句是加分点）。 - M 步用**累加求和再除计数**，而不是"每次平均"——避免除法的累积误差，也避免空簇除零。 - `if (cnt[j] == 0) continue;`：**空簇处理**，保留旧中心。这是面试官最爱问的实现细节。 - 收敛判据用**中心移动平方和**（shift），不是标签是否变化（标签判据可能出现标签循环震荡）。 - `iters=100` 只是保险，正常迭代次数远小于它。

追问 Q&A： - **Q：k 怎么选？** A：肘部法（inertia 随 k 的拐点）、轮廓系数 silhouette（−1~1，越大越好）、Gap statistic；实际业务常直接用下游指标定 k。 - **Q：kmeans++ 是什么？** A：初始化时第一个中心随机，之后每个新中心按"到已有中心的

最近距离平方"成比例采样——离得远的点更容易被选，直接把随机初始化最坏情况压下去。 - **Q：一定会收敛吗？收敛到全局最优吗？** A：目标函数（inertia）每步单调不增且有下界，所以一定收敛；但**只保证局部最优。这条我有实测数据**：在我自测的三高斯簇（150 点）上，seed=7 单次初始化只对上 125/150，换 20 个初始化取最好就 150/150——可以直接把这个数字讲给面试官，说服力极强。 - **Q：kmeans 的假设/局限？** A：簇是凸的、各向同性、方差相近、大小相近；对异常值敏感（均值被拉走，可用 k-medoids 或剪枝）；只给硬分配（软分配要 GMM/EM）；高维失效（距离集中现象）。 - **Q：复杂度与加速？** A：O(n · k · d · iters)；加速：MiniBatch kmeans（每批采样更新）、Elkan 三角不等式剪枝、kd-tree/球树（低维）、先 PCA 降维。 - **Q：kmeans 和 GMM 的关系？** A：GMM 用 EM 做软分配 + 各簇自己的协方差；kmeans 是"等协方差 + 硬分配"的特例。

现场最易错 3 点：用 `vector<vector<float>>` 存中心导致下标混乱；空簇除零；收敛判据用标签导致死循环。

2.6 Self-Attention（多模态岗最核心的一题，占分最高）

面试官原话："手写一下 self-attention" → 通常紧跟三个追问：为什么要除以 √d、复杂度多少、mask 怎么加。

30 秒口述脚本（先说，让对方知道你要写什么）：

> "self-attention 是四步：X 分别乘 Wq/Wk/Wv 得到 Q、K、V；Q 乘 K 的转置得到 L×L 的相似度矩阵，除以 √d 缩放；遮掉 mask 位置后逐行 softmax 得到注意力权重；最后权重乘 V 得到输出。输出形状和输入一样是 L×D。"

公式：

```
Attention(Q, K, V) = softmax( Q · K^T / sqrt(d_k) + mask ) · V
    Q = X · Wq    K = X · Wk    V = X · Wv      (X: L×D, W*: D×D)
    Q · K^T : L×L        softmax 按行做        结果 : L×D
```

代码（这段是完整可跑版，来自 `ai_interview_cpp.cpp`）：

```cpp
// X: L x D, Wq/Wk/Wv: D x D, mask: L x L (true=屏蔽) 可为空
vector<float> selfAttention(const vector<float>& X, int L, int D,
                            const vector<float>& Wq, const vector<float>& Wk,
                            const vector<float>& Wv, const vector<float>& mask,
                            vector<float>* attnOut = nullptr) {
    auto Q = matmul(X, Wq, L, D, D);            // [L,D]
    auto K = matmul(X, Wk, L, D, D);
    auto V = matmul(X, Wv, L, D, D);

    float scale = 1.0f / sqrtf((float)D);       // 缩放，防止点积过大把 softmax 推饱和
    vector<float> S(L * L, 0.f);
    for (int i = 0; i < L; ++i)
        for (int j = 0; j < L; ++j) {
            float s = 0;
            for (int t = 0; t < D; ++t) s += Q[i * D + t] * K[j * D + t];
            S[i * L + j] = s * scale;           // = q_i · k_j / sqrt(d)
        }
    if (!mask.empty())
        for (int i = 0; i < L * L; ++i)
            if (mask[i]) S[i] = -1e30f;         // 屏蔽位：用一个极大的负数，exp 后为 0
    for (int i = 0; i < L; ++i) softmaxInPlace(S, i * L, L);   // 逐行 softmax
    auto O = matmul(S, V, L, L, D);             // [L,L] x [L,D] -> [L,D]
    if (attnOut) *attnOut = S;
    return O;
}
```

逐行拆解（这段一定要能讲，面试官几乎必问）：

步骤	代码	讲法
1. 三次投影	`matmul(X, Wq, L, D, D)`	X 是 L 个 token、每个 D 维；乘 D×D 权重后还是 L×D。**三个投影矩阵不共享**，这是 self-attention 有"三种角色"的原因
2. 打分	三层循环 `Q[i*D+t] * K[j*D+t]`	取 Q 的第 i 行、K 的第 j 行，逐维乘加 = 点积。注意是 `K[j*D+t]` 而不是 `K[t*D+j]` ——**这里等价于 Q · K^T**，因为 K 的第 j 行就是 K^T 的第 j 列
3. 缩放	`* scale`	见下面追问，方差论证
4. mask	`S[i] = -1e30f`	加性 mask：加一个极负的数，exp 后趋 0，相当于"看不到"

步骤	代码	讲法
5. 逐行 softmax	`softmaxInPlace(S, i*L, L)`	每一行（一个 query 对所有 key 的注意力）单独归一化。**这正是我 2.1 把 off/len 抽出来的原因**
6. 加权求和	`matmul(S, V, L, L, D)`	L×L 的权重乘 L×D 的 V → L×D，第 i 行输出 = Σ_j attn[i][j] · V[j]

追问 Q&A（按被问概率排序）：

- **Q：为什么除以 √d_k?** （必问，答不出直接掉档） A：假设 q、k 各维独立、均值 0 方差 1，则点积 `q·k = Σ_t q_t k_t` 的方差是 d_k，标准差是 √d_k。d_k 大（比如 64、128）时点积量级到 10 以上，softmax 会被推到饱和区：最大项接近 1、其余接近 0，梯度几乎为 0（softmax 饱和）。除以 √d_k 把方差拉回 1，softmax 落在梯度良好的区间。
- **Q：复杂度?** A：时间 $O(L^2 \cdot D)$（打分和加权各一次），显存 $O(L^2)$（要存注意力矩阵）。L 是序列长度，所以长上下文是平方级代价。
- **Q：$O(L^2)$ 怎么省?** （必答，直接连到你熟悉的领域） A：四条线——① **FlashAttention**：分块（tiling）+ 在线 softmax，把 L×L 矩阵切块流式计算，**省的是显存和 HBM 读写，不减少 FLOPs**；② **稀疏/滑窗注意力**（Longformer、BigBird）；③ **低秩/线性注意力**（Linformer、Performer 用核函数近似）；④ **MQA/GQA**（共享 K/V 头，主要省推理时的 KV cache）；⑤ 状态空间模型 Mamba（线性复杂度，我做过多任务模型里的时序模块，可以顺势讲）。
- **Q：mask 有哪几种?** A：① **causal mask**（下三角可见，上三角屏蔽）用于自回归生成；② **padding mask**（按 key 的有效长度屏蔽 pad）；③ **结构 mask**（跨模态/跨样本隔离）。实现统一是加 −∞ 或 −1e30。
- **Q：softmax 里一行全被 mask 会怎样?** A：分母为 0 → NaN。工程上要么保证每行至少一个有效位，要么对全屏蔽行单独置 0。
- **Q：self-attention 和 cross-attention 区别?** A：Q 的来源。self 是 Q/K/V 同源；cross 是 Q 来自解码器（或 query token），K/V 来自编码器（或另一种模态）。**多模态融合就是 cross-attention 的典型**——图像 token 当 K/V、文本 token 当 Q（或反过来）。这条一定主动讲，能把你简历里的视觉-激光融合接上。
- **Q：为什么要有位置编码?** A：self-attention 对 token 顺序是置换不变的（打乱输入只是打乱输出行），本身不含位置信息。方案：正弦/可学习绝对编码、RoPE（旋转位置编码，保相对位置、可外推，LLaMA 系）、ALiBi（按距离线性衰减 bias）。
- **Q：多头和单头在计算量上一样吗?** A：**几乎一样**。总维度 D 被切成 h 份，每份 d_k=D/h，注意力计算量 $h \cdot L^2 \cdot (D/h) = L^2 \cdot D$，与单头相同；差别只在"分成 h 个子空间分别学依赖"。
- **Q：KV cache 占多少显存?** A：每层每 token 要存 K 和 V 各 D 个值，即 2·D；总显存 ≈ 2 · 层数 · D · 序列长度 · batch · 精度字节。这就是长上下文推理的瓶颈，也是 MQA/GQA 的动机。

现场最易错 3 点： 1. 写成 `K[t*D+j]`（那是 K 而不是 K^T 的取法，维数甚至对不上）； 2. 忘了逐行 softmax 直接对整块做（会把整个 L×L 归一化成和 1）； 3. mask 用 `0` 表示屏蔽（0 加进去等于没屏蔽，exp(0)=1）。

2.7 Multi-Head Attention（承接上题，通常让你"改成多头"）

面试官原话："在上面基础上改成多头" / "多头的 Q、K、V 是怎么切分的？"

30 秒口述脚本：

> "多头就是把 D 维按**列**切成 h 份，每份 d_k = D/h。第 i 个头用 Wq/Wk/Wv 的第 i 个列块做投影，得到 L×d_k 的 Q/K/V，各自算一遍 attention，然后把 h 个 L×d_k 的输出按列拼回 L×D，最后过一遍输出投影 Wo。参数量是 $4D^2$。"

公式：

```
head_i = Attention(X·Wq[:, i·dk:(i+1)·dk], X·Wk[:, 同列块], X·Wv[:, 同列块])
MHA(X) = Concat(head_0, ..., head_{h-1}) · Wo
```

代码（完整可跑，来自验证文件）：

```
vector<float> multiHeadAttention(const vector<float>& X, int L, int D, int h,
                                 const vector<float>& Wq, const vector<float>& Wk,
                                 const vector<float>& Wv, const vector<float>& Wo,
                                 const vector<float>& mask,
                                 vector<float>* attnOut = nullptr) {
```

```
    int dk = D / h;
    vector<float> Ocat(L * D, 0.f);
    vector<float> attnAll(h * L * L, 0.f);
    for (int hi = 0; hi < h; ++hi) {
        // 第 hi 个头只取权重的第 [hi*dk, hi*dk+dk) 列
        auto headW = [&](const vector<float>& W) {
            vector<float> Ws(D * dk);
            for (int r = 0; r < D; ++r)
                for (int c = 0; c < dk; ++c) Ws[r * dk + c] = W[r * D + hi * dk + c];
            return Ws;
        };
        auto Q = matmul(X, headW(Wq), L, D, dk);
        auto K = matmul(X, headW(Wk), L, D, dk);
        auto V = matmul(X, headW(Wv), L, D, dk);
        float scale = 1.0f / sqrtf((float)dk);
        vector<float> S(L * L, 0.f);
        for (int i = 0; i < L; ++i)
            for (int j = 0; j < L; ++j) {
                float s = 0;
                for (int t = 0; t < dk; ++t) s += Q[i * dk + t] * K[j * dk + t];
                S[i * L + j] = s * scale;
            }
        if (!mask.empty())
            for (int t = 0; t < L * L; ++t) if (mask[t]) S[t] = -1e30f;
        for (int i = 0; i < L; ++i) softmaxInPlace(S, i * L, L);
        auto O = matmul(S, V, L, L, dk);
        for (int i = 0; i < L; ++i)                       // 拼回第 hi 头占的那几列
            for (int c = 0; c < dk; ++c) Ocat[i * D + hi * dk + c] = O[i * dk + c];
        copy(S.begin(), S.end(), attnAll.begin() + (size_t)hi * L * L);
    }
    if (attnOut) *attnOut = attnAll;
    return matmul(Ocat, Wo, L, D, D);                     // 最后过输出投影
}
```

逐行拆解：

- `int dk = D / h`：每个头的维度。**面试时先问一句"h 必须整除 D 吧"**，这是在展示你有工程感。
- `headW`：从 D×D 权重里切出 D×dk 的子块。**关键点：切的是列**。解释："Wq 的每一列对应输出的一维。第 hi 个头管输出的 [hi·dk, (hi+1)·dk) 这几维，所以取这段列。"
- `matmul(X, ..., L, D, dk)`：X 是 L×D、子权重 D×dk → 输出 L×dk。**尺寸参数必须同步改成 dk**，这是最容易错的地方。
- `Ocat[i * D + hi * dk + c]`：往 L×D 的大矩阵里"按列拼"。第 i 个 token 输出的第 (hi·dk + c) 维，就是第 hi 头输出的第 c 维。
- `attnAll`：把所有头的注意力矩阵存一起，shape h×L×L，方便可视化不同头看什么。
- 最后 `matmul(Ocat, Wo, L, D, D)`：投影回 D 维。**Wo 不能忘**，这是"多头之间做信息混合"的唯一机会。

我在本地验证过的三个性质（可以作为"我怎么确认写对了"的答案讲出来）： 1. `h=1 且 Wo=单位阵` 时，多头实现和单头实现**逐元素完全一致**（数值误差 1e-5 内）——这是最强的自检手段。 2. 每个头、每一行的 attention 权重和恒为 1。 3. 输出 shape 恒为 L×D，与输入一致（这是能堆叠 N 层的前提）。

追问 Q&A： - **Q：参数量？** A：Wq/Wk/Wv/Wo 各 D×D，共 **$4D^2$**（加 bias 再 +4D）。**和头数 h 无关**——这是常被考的常识。 - **Q：为什么多头比单头好？** A：不同头在不同子空间学不同的依赖模式（位置邻近、语法、指代、跨模态对应），类似 CNN 的多通道分工；单头只有一套注意力分布。 - **Q：MQA / GQA 是什么？省在哪？** A：MQA 让所有头共享同一份 K/V（h 个头只有 1 组 K/V）；GQA 折中，h 个头分 g 组、每组共享一组。省的是**推理时的 KV cache**：MHA 每 token 存 2D 个数，MQA 存 2D/h，GQA 存 2D·g/h。训练 FLOPs 基本不变。 - **Q：为什么 Q、K、V 用不同权重？** A：三者角色不同——Q 是"我要找什么"，K 是"我能被什么匹配到"，V 是"被匹配上之后提供什么内容"。同权重会限制表达。 - **Q：C++ 里"拆头"和 PyTorch 里 `view + transpose` 是什么关系？** A：同一件事。PyTorch 是 `[B,L,D] → view(B,L,h,dk) → transpose(1,2) → [B,h,L,dk]`，C++ 这里是直接在权重上按列切、输出按列拼，等价于 transpose 后连续化。**如果面试官追问 PyTorch 版，我按 4.2 节那样写给他**（这句是兜底话术，背下来）。 - **Q：注意力头之间真的分工不同吗？有证据吗？** A：可视化证据很多（有的头盯相邻 token、有的盯句法依存、有的盯罕见词/分隔符），也有研究报告存在冗余头（可剪枝）——答"部分是冗余的，所以有 head pruning 这类工作"很稳。

现场最易错 3 点：`dk` 忘了替换进 `matmul` 的尺寸参数；拼回时用 `hi*dk*c` 之类的错误偏移；忘了乘 Wo。

2.8 CART 分类树（传统 ML 基础题，代码最长的）

面试官原话："手写一个决策树" / "CART 的划分准则是什么" / 然后追一串：剪枝、连续特征、和 ID3/C4.5 的区别、GBDT 怎么从它推出来。

30 秒口述脚本：

> "CART 是二叉树：每次在所有特征的所有候选阈值里，选让加权基尼不纯度下降最多的那个 (特征, 阈值) 做二分切分，然后递归到左右子集。停止条件是深度到上限、样本太少、或节点已纯。叶子取多数类。"

公式：

```
基尼不纯度   Gini(S) = 1 - Σ_c (|S_c| / |S|)²
切分增益     Gain = Gini(S) - (|S_L| · Gini(S_L) + |S_R| · Gini(S_R)) / |S|
候选阈值     相邻不同取值的中点 (v_p + v_{p+1}) / 2
```

代码（完整可跑；`pool` 是节点池，用 `int` 下标代替指针，避免手写内存管理）：

```cpp
struct TreeNode { int feat = -1; float thr = 0; int left = -1, right = -1; int label = 0; };

double gini(const vector<int>& cnt, int n) {
    if (n <= 0) return 0.0;
    double s = 0;
    for (int c : cnt) { double p = (double)c / n; s += p * p; }
    return 1.0 - s;
}

int buildTree(vector<TreeNode>& pool, const vector<float>& X, const vector<int>& y,
              const vector<int>& idx, int d, int K, int depth, int maxDepth, int minLeaf) {
    int id = (int)pool.size();
    pool.push_back(TreeNode());

    vector<int> cnt(K, 0);
    for (int i : idx) ++cnt[y[i]];
    pool[id].label = (int)(max_element(cnt.begin(), cnt.end()) - cnt.begin());   // 叶子取值 = 多数类

    if (depth >= maxDepth || (int)idx.size() < 2 * minLeaf) return id;           // 停止条件 1/2
    double parent = gini(cnt, (int)idx.size());
    if (parent < 1e-12) return id;                                               // 停止条件 3: 已纯

    double bestGain = 1e-12;
    int bf = -1; float bt = 0;
    int n = (int)idx.size();
    for (int f = 0; f < d; ++f) {
        vector<pair<float, int>> v;
        v.reserve(n);
        for (int i : idx) v.push_back({X[i * d + f], i});
        sort(v.begin(), v.end());                                    // 排序后扫一遍就能拿到所有候选切分的不纯度
        vector<int> lc(K, 0), rc = cnt;
        for (int p = 0; p + 1 < n; ++p) {
            int yi = y[v[p].second];
            ++lc[yi]; --rc[yi];
            if (v[p].first == v[p + 1].first) continue;               // 同值不能切
            int nl = p + 1, nr = n - nl;
            if (nl < minLeaf || nr < minLeaf) continue;
            double g = parent - (nl * gini(lc, nl) + nr * gini(rc, nr)) / n;
            if (g > bestGain) { bestGain = g; bf = f; bt = (v[p].first + v[p + 1].first) * 0.5f; }
        }
    }
    if (bf < 0) return id;

    vector<int> il, ir;
    for (int i : idx) (X[i * d + bf] <= bt ? il : ir).push_back(i);
    pool[id].feat = bf; pool[id].thr = bt;
    pool[id].left = buildTree(pool, X, y, il, d, K, depth + 1, maxDepth, minLeaf);
    pool[id].right = buildTree(pool, X, y, ir, d, K, depth + 1, maxDepth, minLeaf);
    return id;
}

int predictTree(const vector<TreeNode>& pool, const vector<float>& x, int d) {
    int u = 0;
    while (pool[u].feat >= 0) u = (x[pool[u].feat] <= pool[u].thr) ? pool[u].left : pool[u].right;
```

```
    return pool[u].label;
}
```

逐行拆解（这是最容易写崩的一题，重点看下面五条）：

1. **节点池 `pool` + `int` 下标**：`int id = pool.size(); pool.push_back(...)` 先占位，再递归填充 `pool[id].left/right`。用下标而不是 `TreeNode*` 的原因：`push_back` 可能触发扩容，**指针会失效**，下标不会。面试时讲这句"我用下标避免 realloc 后指针失效"，是很硬的工程细节。
2. **三个停止条件**必须都写：`depth >= maxDepth`、`样本数 < 2*minLeaf`（左/右都至少 minLeaf 个才能切）、`parent < 1e-12`（已纯，再切是浪费）。漏第三个会导致无限递归/极深的树。
3. **前缀扫描技巧**：对每个特征，把样本按该特征值排序，然后从左往右扫：每挪动一个样本，把它从右子集搬到左子集（`++lc[yi]; --rc[yi];`），此时左右两侧的类别计数是 O(1) 更新，不用每次重新统计。**总复杂度从 $O(n^2 \cdot d)$ 降到 $O(n \cdot d \cdot \log n)$（排序主导）**。这一段是本题最大的加分点，主动讲。
4. **同值不能切**：`if (v[p].first == v[p+1].first) continue;` —— 两个样本特征值相同，任何阈值都分不开它们，切了是无意义分裂。
5. **阈值取中点**：`(v[p].first + v[p+1].first) / 2`，并且预测时用 `<=` 走左，保证阈值语义一致（写 `<=` 就不要在预测里写 `<`）。
6. **重新划分再递归**：确定最优 (bf, bt) 后，用一次线性扫描把 `idx` 分成 `il/ir`。没有在扫描阶段缓存划分结果，是因为那样要为每个特征存一份临时数组，代码更长；面试写"慢一点但清楚"的版本完全 OK，主动说一句"工程上可以缓存"。

追问 Q&A：- **Q：CART / ID3 / C4.5 的区别?**（几乎必问） A：ID3 用**信息增益**、多叉分裂、只能处理离散特征、偏好多取值特征；C4.5 用**信息增益率**（除以特征自身的分裂信息，惩罚多值特征）、支持连续特征与缺失值、仍倾向多叉；CART 用**基尼指数**、**严格二叉**、分类用基尼 / 回归用 MSE、支持剪枝，是 sklearn 和 GBDT 家族的基础。 - **Q：为什么 CART 用基尼不用熵?** A：基尼是 $1 - \Sigma p^2$，只有乘法和加法，没有 log，计算快（大样本下差别明显）；两者曲线形状高度相似，划分结果基本一致。**还有一个细节**：基尼可理解为"随机抽两个样本类别不同的概率"。 - **Q：剪枝怎么做?** A：预剪枝（本代码的三种停止条件）：快，但贪心视野短，可能过早停；后剪枝（代价复杂度剪枝 CCP）：先长满，再自底向上用 $R(T) + \alpha|T|$ 判断是否合并叶子，α 由交叉验证选——更准但更贵。sklearn 的 `ccp_alpha` 就是它。 - **Q：为什么树容易过拟合?** A：分裂太细，叶子只剩 1 个样本 → 方差爆炸；阈值微动就换一棵树（不稳定）。缓解：限制深度/叶最小样本/后剪枝/集成（RF 降方差、GBDT 降偏差）。 - **Q：连续特征怎么处理?** A：排序 + 取相邻中点（本代码）；大样本用分位分桶（LightGBM 的直方图，把候选阈值从 n 个降到 255 个桶）。 - **Q：特征重要性怎么算?** A：把每次分裂带来的"加权不纯度下降 × 该节点样本数"按特征累加，再归一化（sklearn 的 `feature_importances_`）。**缺点**：偏好多取值特征，对相关特征会分摊；更可靠的是置换重要性（打乱该列看指标降多少）。 - **Q：从 CART 怎么推到 GBDT / XGBoost?**（面到这一步说明前面都过了） A：GBDT = 串行加法模型，每棵新树拟合当前模型的**负梯度（残差）**，实现"用树做梯度下降"；XGBoost 在此基础上：① 用损失函数的**二阶泰勒展开**（一阶梯度 g、二阶 h）；② 目标函数显式加正则 $\gamma T + \lambda/2 \cdot \Sigma w^2$；③ 最优叶权重有闭式解 $w^* = -\Sigma g / (\Sigma h + \lambda)$，分裂增益有解析式；④ 工程上直方图 + 特征预排序 + 并行。LightGBM 换成 leaf-wise 生长 + GOSS（梯度采样）+ EFB（互斥特征捆绑）。 - **Q：回归树和分类树的区别?** A：划分准则从基尼换成**MSE 下降**，叶子值取均值而不是众数；其余（二分、递归、贪心阈值搜索）完全一样。 - **Q：随机森林和 GBDT 的区别?** A：RF = bagging（并行、样本+特征双采样、投票），主要降**方差**；GBDT = boosting（串行、拟合残差），主要降**偏差**。

现场最易错 3 点：漏停止条件导致栈溢出；索引池用指针被 realloc 打崩；候选阈值没跳过同值导致无效分裂。

---

3. 力扣部分（C++ 版）

3.1 华为机试形态（决定你练什么）

- 校招机试：**3 题 / 400 分 / 120 分钟**，约 简单100 + 中等200 + 困难300，**ACM 模式**（自己读标准输入、自己打印），可用本地 IDE 调试。**做出 200 分就进面**，第一题必须稳。
- 技术面手撕：面试官给题、你共享屏幕写，通常 LeetCode 中等。要求"能跑 + 说复杂度"。
- 华为偏好：**字符串模拟、DFS/BFS、DP、二叉树、双指针**。图论进阶（网络流、线段树）几乎不考。
- 语言：机试支持 C/C++，你直接用 C++，不用换。

3.2 IO 模板（很多人挂在这，不是挂算法）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);   // 关掉 C 与 C++ 流同步，提速
    cin.tie(nullptr);              // 解绑 cin/cout，提速

    // 场景 1：第一行 n，之后 n 个整数
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    long long sum = 0;
    for (int i = 0; i < n; ++i) { cin >> a[i]; sum += a[i]; }
    cout << sum << "\n";

    // 场景 2：多组数据直到 EOF（华为常见）
    // int a, b; while (cin >> a >> b) cout << a + b << "\n";

    // 场景 3：一行里用逗号分隔 -> 先读成字符串再切
    // string line; getline(cin, line);
    // stringstream ss(line); string tok;
    // while (getline(ss, tok, ',')) v.push_back(stoi(tok));

    // 场景 4：先读表头行再读数据（cin >> 与 getline 混用必须先吃掉换行）
    // cin >> n; cin.ignore(); getline(cin, line);
    return 0;
}
```

三条铁律：1. `cin >>` 和 `getline` **不要混着用**，混了就先 `cin.ignore()` 吃掉残留换行——这是 ACM 模式第一大坑。2. 多组数据一律写 `while (cin >> x)`，别假设只有一组。3. 输出用 `"\n"` 不用 `endl`（`endl` 强制 flush，数据量大时拖慢）。

3.3 七个模板（现场默写）

```cpp
// T1 快排（随机化 pivot，避免有序数组退化）
void quickSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int p = l + rand() % (r - l + 1);
    swap(a[l], a[p]);
    int pivot = a[l], i = l;
    for (int j = l + 1; j <= r; ++j) if (a[j] < pivot) swap(a[++i], a[j]);
    swap(a[l], a[i]);
    quickSort(a, l, i - 1);
    quickSort(a, i + 1, r);
}

// T2 二分（左边界 / lower_bound 手写版）
int lowerBound(const vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size();          // 注意 hi 是 size 而不是 size-1（左闭右开）
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;        // 防溢出写法
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// T3 滑动窗口（无重复字符最长子串）
int lengthOfLongestSubstring(const string& s) {
    vector<int> last(128, -1);
    int left = 0, ans = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        unsigned char c = s[i];
        if (last[c] >= left) left = last[c] + 1;
        last[c] = i;
        ans = max(ans, i - left + 1);
    }
    return ans;
}

// T4 优先队列 TopK（n log k）
vector<int> topK(const vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;   // 小顶堆
```

```
    for (int x : nums) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> res;
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    return res;
}

// T5 并查集（路径压缩 + 按秩合并）
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (r[ra] < r[rb]) swap(ra, rb);
        p[rb] = ra;
        if (r[ra] == r[rb]) ++r[ra];
        return true;
    }
};

// T6 二叉树层序（BFS 模板，所有"层"类题都从它改）
struct TNode { int val; TNode *left = nullptr, *right = nullptr; };
vector<vector<int>> levelOrder(TNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();                    // 关键：先固定这一层的节点数
        vector<int> layer;
        while (sz--) {
            TNode* u = q.front(); q.pop();
            layer.push_back(u->val);
            if (u->left) q.push(u->left);
            if (u->right) q.push(u->right);
        }
        res.push_back(layer);
    }
    return res;
}

// T7 01 背包（一维滚动数组，内层必须倒序）
int knapsack01(const vector<int>& w, const vector<int>& v, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int c = W; c >= w[i]; --c)       // 倒序：保证每个物品只用一次
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

为什么 01 背包要倒序：正序时 `dp[c-w[i]]` 可能是本轮刚被更新的值（= 同一物品用了多次，退化成完全背包）；倒序保证用的是上一轮的值。

3.4 STL 速查（只列现场真正用到的 20 条）

需求	写法
数组	`vector<int> a(n, 0);`
二维	`vector<vector<int>> g(n, vector<int>(m, 0));`
排序降序	`sort(a.begin(), a.end(), greater<int>());`
自定义排序	`sort(v.begin(), v.end(), [](auto& x, auto& y){ return x.first < y.first; });` **必须严格弱序**
去重	`sort` 后 `a.erase(unique(a.begin(), a.end()), a.end());`
查找	`binary_search` / `lower_bound` / `upper_bound` （**前提：已排序**）
最大/最小	`*max_element(a.begin(), a.end())` （返回迭代器！）

需求	写法
求和	`accumulate(a.begin(), a.end(), 0LL)`（**初值用 0LL 防溢出**）
计数	`count(a.begin(), a.end(), x)`
反转	`reverse(s.begin(), s.end());`
填充	`fill(dp.begin(), dp.end(), 0);`
字符串转数	`stoi / stoll / stod`（大数用 `stoll`），`to_string(x)`
字符判断	`isdigit(c)`、`isalpha(c)`、`toupper(c)`
哈希表	`unordered_map<int,int> mp; mp[k]++;`（**注意 `mp[k]` 会插入默认值**，查存在用 `count` / `find`）
计数数组（小范围）	`int cnt[26] = {0};`（比 map 快一个量级）
集合有序	`set<int> s; s.insert(x); s.erase(x);`（O(log n)，中位数/前驱后继用它）
小顶堆	`priority_queue<int, vector<int>, greater<int>>`
双向队列	`deque<int> dq; dq.push_front(x);`（单调队列/滑动窗口最大值）
栈	`stack<char> st;`
位运算	`__builtin_popcount(x)`、`x & -x`（lowbit）

3.5 20 题清单（C++ 关键点）

#	题	C++ 关键点
1	两数之和 / 三数之和	`unordered_map`；三数之和先排序 + 双指针 + 跳过重复
2	无重复字符最长子串	`vector<int> last(128,-1)`（别用 unordered_map）
3	最长回文子串	中心扩展 2n-1 个中心
4	有效括号	`stack<char>`，空栈时 `top()` 会崩
5	字符串反转 / 翻转单词 / 大数相加	`string` 拼接；大数用进位 `int c`
6	合并两个有序链表 / 反转链表	哑结点 `ListNode dummy, *t = &dummy;`
7	环形链表 / 相交链表	快慢指针；相交走两遍换头
8	LRU 缓存	`list<pair<int,int>>` + `unordered_map<int, list<...>::iterator>`，**注意 list 迭代器在 splice 后仍有效**
9	二叉树层序 / 最大深度 / 之字形	BFS 先存 `sz = q.size()`；之字形用 `reverse` 隔层
10	验证 BST	传上下界 `long long lo, hi`（**节点值可能是 INT_MIN/MAX**）
11	岛屿数量 / 最大面积	DFS 淹没，或 DSU；大输入注意递归深度
12	腐烂的橘子	多源 BFS：所有源先入队，再按层计数
13	全排列 / 子集 / 组合总和	`sort` + 回溯 + 同层跳过 `i>start && a[i]==a[i-1]`
14	爬楼梯 / 打家劫舍	`long long` 防溢出（70 题会超 int？不会，但要养成习惯）
15	编辑距离 / LCS	二维 dp，`vector<vector<int>>`，注意初始化第 0 行/列
16	最大子数组和	Kadane，`int cur = 0, best = a[0]`
17	第 K 大 / 前 K 高频	小顶堆；高频用 `unordered_map` 计数再堆
18	搜索旋转排序数组	二分时先判断哪半边有序
19	零钱兑换	完全背包：外层物品内层容量**正序**

#	题	C++ 关键点
20	手写快排 / 归并 / 堆排	见 3.3；归并要求辅助数组

3.6 C++ 现场易错 8 条（每条都真的会挂）

1. **溢出**：`int` 相乘/累加超 2e9 直接 UB。看到"和/乘积/路径长度"就先想 `long long`。
2. **`size()` 是无符号**：`for (int i = 0; i < a.size() - 1; ++i)` 在 `a` 为空时 `size()-1` 变成巨大正数 → 死循环。写法：`int n = (int)a.size();` 先转 int。
3. **`sort` 的比较函数必须严格弱序**：写 `<=` 或 `>=` 在某些 STL 实现下直接越界崩溃。
4. **`unordered_map` 用 `[]` 会插入默认值**：判断存在用 `count` 或 `find`，别用 `mp[k] != 0`。
5. **迭代器失效**：`vector` 在 `push_back` 后旧迭代器/指针可能失效；`unordered_map` 在 rehash 后所有迭代器失效（`list` 的插入不失效）。
6. **`cin >>` 后接 `getline`**：必须先 `cin.ignore()`。
7. **递归深度**：DFS 在 1e5 级别链式数据上会爆栈，改成显式 `stack` 或迭代。
8. **多组数据忘了重置全局数组**：`fill` 一遍，别靠默认初始化。

---

4. 万一面试官坚持要 Python / PyTorch

4.1 先说三句兜底话术（背下来）

1. "核心逻辑我刚才用 C++ 展开写完了，PyTorch 版是把循环换成张量算子，我写一下映射关系。"
2. "权重投影就是 `nn.Linear`，拆头用 `view + transpose`，打分是 `Q @ K.transpose(-2,-1)`，缩放除 `sqrt(d_k)`。"
3. "我平时主力是 C++/工程侧，PyTorch 我读得懂、改得动，写这种标准实现没问题。"（**不说"我不会 Python"，说"我能写、只是首选 C++"**）

4.2 三张对照表（C++ ↔ numpy ↔ torch）

要做的事	C++	numpy	torch
建矩阵 n×m 全零	`vector<float> A(n*m, 0.f)`	`np.zeros((n, m))`	`torch.zeros(n, m)`
取元素 (i,j)	`A[i*m + j]`	`A[i, j]`	`A[i, j]`
矩阵乘	`matmul(A,B,n,k,m)`	`A @ B`	`A @ B`
逐元素乘	`for` 循环	`A * B`	`A * B`
按行求和	`for` 累加	`A.sum(-1, keepdims=True)`	`A.sum(dim=-1, keepdim=True)`
按行最大值	`for` 找 max	`A.max(-1, keepdims=True)`	`A.max(dim=-1, keepdim=True).values`
softmax（按行）	手写三段循环	手写 `e = np.exp(A - A.max(-1, keepdims=True)); e / e.sum(-1, keepdims=True)`	`torch.softmax(A, dim=-1)`
形状变换	无（本来就是扁平的）	`A.reshape(n, h, dk)`	`A.view(n, h, dk)` / `.reshape`
转置后两维交换	索引里换位置	`A.transpose(1, 2)`	`A.transpose(1, 2)`
生成全 1 / 单位阵	`vector<float>(n,1.f)`	`np.ones(n)` / `np.eye(n)`	`torch.ones(n)` / `torch.eye(n)`
取一列	`A[i*m + j]`	`A[:, j]`	`A[:, j]`
均值 / 方差	手动累加再除	`A.mean(-1)` / `A.var(-1)`	`A.mean(-1)` / `A.var(-1, unbiased=False)`

要做的事	C++	numpy	torch
屏蔽赋值	`if (mask[i]) S[i] = -1e30f;`	`np.where(mask, -1e9, S)`	`S.masked_fill(mask, float('-inf'))`
拼回多头	`Ocat[i*D + hi*dk + c] = ...`	`np.concatenate` / 先分配再赋值	`.transpose(1,2).contiguous().view(B,L,D)`
随机初始化	`mt19937`	`np.random.default_rng(0)`	`torch.manual_seed(0)`

4.3 三段可直接背的 PyTorch 版

（1）MHA（面试要 Python 时写这段，40 行内）

```python
import math, torch, torch.nn as nn

class MHA(nn.Module):
    def __init__(self, d_model, n_head):
        super().__init__()
        assert d_model % n_head == 0
        self.h, self.dk = n_head, d_model // n_head
        self.wq, self.wk, self.wv, self.wo = [nn.Linear(d_model, d_model) for _ in range(4)]

    def forward(self, q, k, v, mask=None):
        B, Lq, D = q.shape
        Lk = k.size(1)
        Q = self.wq(q).view(B, Lq, self.h, self.dk).transpose(1, 2)   # [B,h,Lq,dk]
        K = self.wk(k).view(B, Lk, self.h, self.dk).transpose(1, 2)
        V = self.wv(v).view(B, Lk, self.h, self.dk).transpose(1, 2)
        s = Q @ K.transpose(-2, -1) / math.sqrt(self.dk)              # [B,h,Lq,Lk]
        if mask is not None:
            s = s.masked_fill(mask.bool(), float('-inf'))             # True = 屏蔽
        a = torch.softmax(s, dim=-1)
        o = (a @ V).transpose(1, 2).contiguous().view(B, Lq, D)       # 拼头
        return self.wo(o), a
```

对照讲：`view(B,Lq,h,dk).transpose(1,2)` 就是 C++ 里"按列切头"；`contiguous().view(B,Lq,D)` 就是 C++ 里的 `Ocat` 拼列。**`contiguous()` 不能省**：transpose 后内存不连续，`view` 会报错，这是面试官最爱抓的一点。

（2）softmax + 交叉熵（要"手写不用内置"时）

```python
def softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)

def cross_entropy(logits, target):                # logits: [N,C]  target: [N]
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    return (lse - logits.gather(1, target[:, None]).squeeze(1)).mean()
```

（3）kmeans（numpy 向量化，10 行）

```python
import numpy as np

def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)].copy()
    for _ in range(iters):
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)     # n×k 距离矩阵
        lab = d.argmin(1)
        newC = C.copy()
        for j in range(k):
            m = lab == j
            if m.any():                                        # 空簇保留旧中心
                newC[j] = X[m].mean(0)
        if np.allclose(newC, C): break
        C = newC
    return lab, C
```

> 说明：C++ 版全部实测通过；上面 Python 代码的对等逻辑（同构的 numpy 实现）也已验证，但这台机器没有 torch 环境，**torch 三段建议你在自己环境跑一遍**再上场。

---

5. 三天练习法（C++ 版，针对"能看不能写"）

5.1 每一天只做"不看书写出来"

天	任务	验收标准（必须逐条对上）
D1 上午	softmax + 交叉熵 + LayerNorm	编译过；softmax 对 1e3 输入不出 nan；LN 输出 mean≈0/var≈1；能答"为什么减 max""为什么除 √d""LN 和 BN 区别"
D1 下午	IoU + NMS	能算手写例子 81/119；能说 Soft-NMS 三个改进
D2 上午	kmeans	三簇能分开；能答"k 怎么选""为什么局部最优（举我给的 125/150 vs 150/150 实测）""空簇怎么办"
D2 下午	self-attention → 改多头	**15 分钟内**单头写完；改多头只改 4 处（dk、权重切片、尺寸参数、拼回）；能答 √dk 的方差论证、FlashAttention 省什么
D3 上午	CART	二分类训练集能到 100%；能答 CART/ID3/C4.5 区别、剪枝两种、怎么推到 GBDT
D3 下午	力扣 4 题（C++，ACM 模式）	用 3.2 模板读入；每题先说复杂度再写；写完自己造 3 个边界用例

5.2 自检清单（面试前一晚过一遍，全部打勾才算准备好）

- [ ] 能默写 `matmul` 的三层循环和三个索引表达式
- [ ] 能一口气说出"softmax 三步 + 为什么减 max"
- [ ] 能说清 Q/K/V 各自的角色，以及为什么除以 √d_k（方差论证）
- [ ] 能说清 self-attention 的时间/显存复杂度，以及 FlashAttention 省的是什么（**省 IO 不省 FLOPs**）
- [ ] 能说清多头"按列切、按列拼"，h=1 时退化为单头（我实测一致）
- [ ] 能列出 mask 的三种类型，并说出"全屏蔽行会 NaN"
- [ ] 能说 CART 用基尼、ID3 用信息增益、C4.5 用增益率，以及为什么 CART 是二叉树
- [ ] 能说出 kmeans 三个局限 + kmeans++ + 空簇处理
- [ ] 能说 NMS 的三个改进方向
- [ ] 力扣：能默写快排/二分/滑窗/优先队列/DSU 五个模板，且知道 01 背包为什么倒序
- [ ] 能说出 C++ 现场 8 条易错里的 5 条（尤其 `size()` 无符号、`long long` 溢出）

5.3 一个加速技巧：把代码"压缩成骨架"

真正要背的不是完整代码，而是骨架。以 MHA 为例，骨架只有 6 行：

```
Q,K,V = XWq, XWk, XWv                        (L×D)
S = Q·K^T / √dk      -> L×L
S = mask(S)                                   (屏蔽位置 -1e30)
A = rowSoftmax(S)                             (逐行归一)
O = A·V                                        (L×D)
Out = Concat(各头 O) · Wo
```

这 6 行能背下来，代码现场就能一行一行"翻译"出来。其余五题同理，我在每题的"口述脚本"里已经给了这个骨架，练习时按骨架复述、不要背整段代码。

附录：配套可运行文件与验证结果

文件：`ai_interview_cpp.cpp`（与本指南同目录，纯 STL，无第三方依赖）

```
g++ -O2 -std=c++17 ai_interview_cpp.cpp -o ai && ./ai
```

本次实测输出（26 项全部 OK）：

模块	通过的断言
matmul	2×3 · 3×2 = [[4,5],[10,11]]
softmax	和为 1；单调性；1e3 输入不溢出；平移不变；与 double 参考值一致（5 项）
交叉熵	与 -log softmax 参考值一致；1e3 级 logits 不溢出（2 项）
LayerNorm	mean≈0、var≈1
IoU / NMS	IoU=81/119；无交集为 0；NMS 保留 {0,2}（3 项）
kmeans	单次 seed=7 一致性 125/150（局部最优实例）；20 次初始化后 150/150
self-attention	行和=1；因果 mask 上三角=0；mask 后行和仍=1；O == A · V（4 项）
multi-head	h=1+Wo=I 与单头逐元素一致；attn 形状 h · L · L；每头行和=1；输出 L×D（4 项）
CART	二分类 ACC=1.0；根阈值 1.670 落在簇间；三分类 ACC=1.0；树确实分裂（4 项）

把 `ai_interview_cpp.cpp` 当练习本用：先在纸上写，再打开文件对答案，最后删掉自己重写一遍。